GP059 Final Year Project Plan: CBOM Scanner (simplified)

Nov 10, 2026 · @Samarth

What we will build

One-line pitch: a command-line tool that scans a software project and a website, lists every place cryptography is used, marks which items will break when quantum computers arrive, and saves the result as a standard CBOM file plus an easy-to-read HTML report.

The problem in plain words. Quantum computers will be able to break RSA and elliptic-curve cryptography, which protect most websites and apps today. Before a company can switch to the new quantum-safe algorithms (NIST's ML-KEM and ML-DSA), it first has to find out where it uses the old ones. That list is called a Cryptographic Bill of Materials (CBOM). Our tool builds that list automatically.

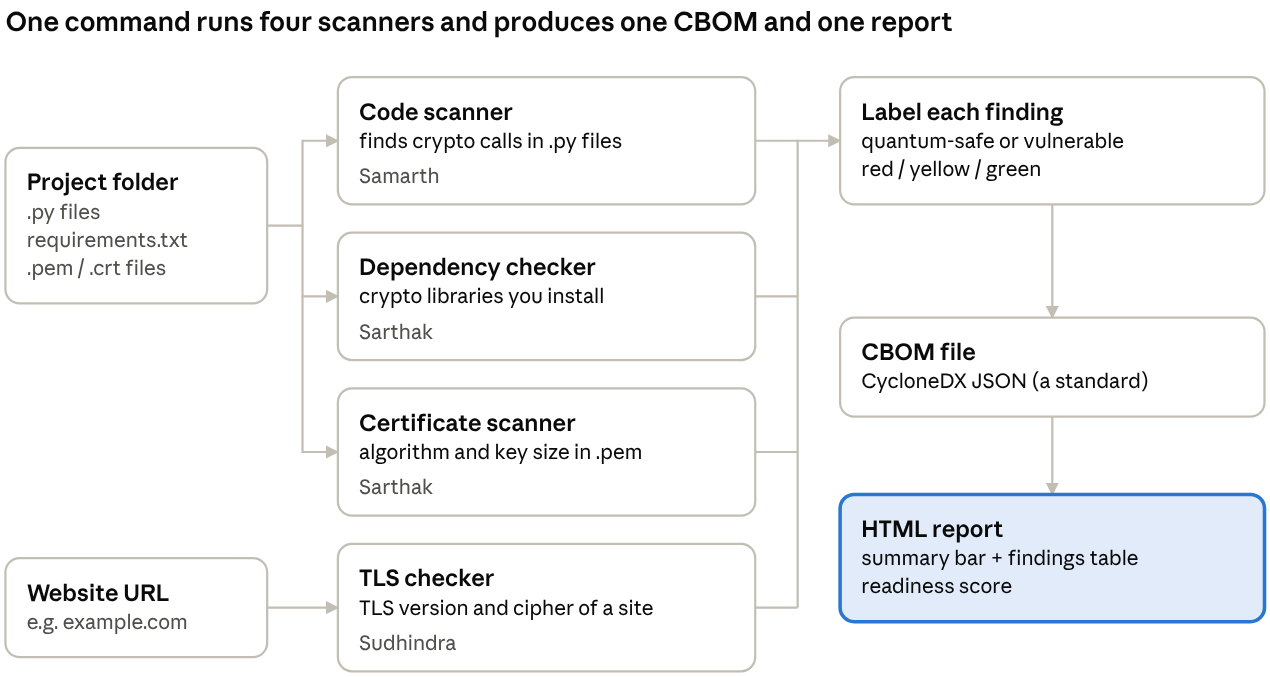
Scope: what we will and will not do. Keeping the scope small is what makes this finishable and easy to demo.

In scope	Out of scope (mention as future work)
Python projects only	Java, C/C++, Go, binaries
Common Python crypto libraries: `hashlib`, `hmac`, `ssl`, `cryptography`, `pycryptodome`, `rsa`, `PyJWT`, `paramiko`	Every library in every language
`requirements.txt` and `pyproject.toml`	Full transitive dependency trees
Certificate and key files (`.pem`, `.crt`, `.key`)	Keystores inside containers or hardware modules
Checking a live website's TLS settings	Scanning the whole internet
CycloneDX CBOM JSON + an HTML report	A full web platform with logins and a database
A simple readiness score (share of assets that are quantum-safe)	Complex risk models

In scope	Out of scope (mention as future work)
Testing on 30 test files we write plus 5 real open-source projects	Large benchmarks and statistics

How it works

The whole tool is four small scanners plus one step that labels and saves the results; each scanner is a few hundred lines of Python.



how the tool works: inputs, four scanners, outputs

The labelling rule is a small lookup table: RSA, DSA, Diffie-Hellman and elliptic-curve (ECDSA, ECDH, Ed25519) are **red** (broken by quantum computers); AES-128, SHA-1 and MD5 are **yellow** (weak today or weakened by quantum); AES-256, SHA-256 and above, ML-KEM and ML-DSA are **green**. Readiness score = green items divided by all items, shown as a percentage.

Libraries you need: `ast` and `ssl` (built into Python), `cryptography`, `cyclonedx-python-lib` (writes the CBOM file), `jinja2` (HTML report), and optionally `sslyze`.

Who does what

Each person owns one or two scanners from start to finish and presents that part in reviews, so every member has something clear to show. Swap names freely.

Person	Builds	Also responsible for
Samarth	**Code scanner:** reads `.py` files with Python's built-in `ast` module and finds crypto calls such as `hashlib.md5()`, `rsa.newkeys(2048)`, `ec.generate_private_key()`	The rules file (which function means which algorithm); report chapter on the code scanner
Sarthak	**Dependency checker** (reads `requirements.txt` / `pyproject.toml`, flags crypto libraries and their versions) and **certificate/key scanner** (opens `.pem` / `.crt` / `.key` files with the `cryptography` library and reads algorithm and key size)	Writing the CycloneDX CBOM file; the readiness score
Sudhindra	**TLS checker:** connects to a website with Python's `ssl` module (or `sslyze`) and records TLS version, cipher suite and certificate algorithm	The HTML report, the single `scan` command that runs everything, GitHub setup
All three	Writing the 30 test files with known answers, testing on real projects, the final report, PPT and demo video	Weekly meeting and progress updates to the guides

Every scanner returns its results in the same simple format, a Python dictionary like `{"file": ..., "line": ..., "algorithm": "RSA", "key_size": 2048, "quantum_safe": False}`. Agree on this format in week 1 so the parts plug together easily.

Timeline

The plan has seven steps and two big checkpoints: a working demo of two scanners at the Project-I review in mid-December, and the full tool with test results at the final viva in late April. Each step takes roughly 3 to 6 hours per person per week, leaving room for other courses.

Seven steps from setup to the final viva, one scanner per person

When	Samarth	Sarthak	Sudhindra
11 to 30 Nov Setup and learning	learn Python ast list crypto functions	learn cryptography lib parse requirements.txt	GitHub repo, folders learn ssl module
1 to 15 Dec **Project-I review**	code scanner v1 hashlib, rsa, AES	dependency checker v1 findings saved as JSON	console output review slides, demo
16 Dec to 10 Jan Exams and break, light	fix review feedback report: introduction	read CycloneDX guide report: background	try sslyze locally report: tools used
11 Jan to 15 Feb Remaining scanners	add PyJWT, paramiko, pycryptodome rules	certificate scanner CycloneDX CBOM file	TLS checker local nginx test server
16 Feb to 15 Mar Report and score	red/yellow/green rules handle edge cases	readiness score validate CBOM schema	HTML report one scan command
16 Mar to 10 Apr Testing	test files: code count hits and misses	test files: certs, deps 5 real projects	TLS checks vs sslyze results tables
11 to 30 Apr **Final submission**	report: method chapter demo rehearsal	report: results chapter final PPT	demo video, README final viva

timeline: 7 steps, Nov 2026 to Apr 2027, tasks per person

The review and viva dates are guesses; move the highlighted rows once the department confirms them. If something slips, cut the optional extras first (the CBOMkit comparison and extra libraries), never the testing step.

What to show at each checkpoint

- **Project-I review (mid-Dec):** the code scanner and dependency checker running on a small sample project and printing findings; the architecture diagram; the timeline above.
- **Mid-term check (mid-Feb, if your department has one):** all four scanners working; a first CBOM file.
- **Final viva (late Apr):** the one-command demo, the HTML report, the CBOM file, the three results tables and the project report.

This week (11 to 17 November)

- [ ] Samarth: write 10 lines of Python that use `ast` to print every function call in a file

- [ ] Sarthak: write a script that reads a `.pem` certificate and prints its algorithm and key size
- [ ] Sudhindra: create the GitHub repository and write a script that prints a website's TLS version and cipher
- [ ] All three: agree on the shared finding format and meet the guides to confirm this reduced scope

How we test it

Testing stays simple: we write small files where we already know the right answer, check whether the tool finds it, and report the result as a table.

1. **30 test files we write ourselves.** Each file uses crypto in one known way: MD5 hash, SHA-256 hash, AES encryption, RSA 2048 key, ECDSA key, a JWT signed with RS256, and so on. We note the correct answer for each in a spreadsheet. Include 5 to 6 "tricky" cases, such as an algorithm name stored in a variable, so the report can honestly show what the tool misses.
2. **Count the results.** For each test file: did the tool find it (correct), miss it, or report something that is not there (false alarm)? From that, compute precision and recall. Both are simple ratios you can explain in one sentence each.
3. **5 real open-source Python projects.** Pick small, well-known ones that use crypto (for example a Flask or Django app with login, or a small SSH tool). Show what the tool found and spot-check 20 findings by hand.
4. **3 TLS checks.** Run the TLS checker on a local test server you configure (for example nginx in Docker with TLS 1.2 and an RSA certificate), and on 2 public websites. Compare with what `sslyze` or the Qualys SSL Labs website reports.
5. **Optional comparison.** If time allows, run IBM's open-source CBOMkit on the same 30 test files and put both tools' results side by side.

The result chapter of the report is then just three tables: test-file results, real-project findings, and TLS checks. Plus one honest paragraph on what the tool cannot detect yet.

Explaining it to evaluators

Evaluators remember a clear story and a live demo far more than technical depth, so prepare both.

The analogy. A food packet has an ingredients list so you can check for allergens. A CBOM is the ingredients list for a software system's cryptography. Our tool reads the system and writes that list, then highlights the "ingredients" that quantum computers will break.

The 3-minute demo script.

1. Show a small sample Python project on screen (one file uses RSA, one uses MD5, one uses SHA-256, plus a `.pem` certificate).
2. Run one command: `python scan.py ./sample-project --url example.com`.
3. Open the HTML report: a summary bar (for example 6 of 10 assets quantum-vulnerable), then a table with each finding, its file and line, and a red, yellow or green label.
4. Open the CBOM JSON file and point out that it follows the CycloneDX standard, so other tools can read it.
5. End with the results table from testing: what the tool found on the 30 test files and the cases it misses.

Questions to rehearse answers for.

- *Why does this matter now?* NIST published the post-quantum standards in 2024 and plans to deprecate RSA/ECC from 2030; companies must first find where they use them.
- *Why is SHA-256 or AES marked safer than RSA?* Quantum attacks (Shor's algorithm) break RSA and elliptic curves completely; symmetric ciphers and hashes are only weakened and stay usable with larger sizes.
- *What is new in your work?* Existing tools usually check only code or only certificates or only websites; ours combines all four in one CBOM and one report.
- *What can't it detect?* Algorithm names built at runtime, crypto inside compiled libraries, and languages other than Python.
- *How did you check it works?* The 30 known-answer test files and spot checks on real projects.

Optional: a short paper

Treat the paper as a bonus after the project works, not a goal that drives it. If the tool and the three result tables are ready by April, the final report already holds 80 percent of a short paper.

The most realistic option is ICISS 2027, an Indian security conference with Springer proceedings; its 2026 deadline was 10 July, so the 2027 one is likely around July, after your viva. Confirm the date when the call appears. Posting the paper on arXiv is a simpler alternative. Ask your guides whether they prefer a specific conference, since they will be co-authors.